

Actividad Asincrónica #7

Brandon Steven Álvarez Franco

Ingeniería en Computación

Instituto Tecnológico de Costa Rica

IC2101 – Programación Orientada a Objetos

Ing. Luis Javier Chavarría Sánchez

B. **Responda en el documento creado en el punto A las siguientes preguntas**:

Sus respuestas deben ser muy claras y suficientemente completas.

1. ¿Por qué razón en el diseño de la super clase Proyecto se incorporó un método abstracto?

Porque todos los proyectos deben calcular su costo actual, pero cada tipo usa una fórmula distinta. La clase general no puede definir una sola fórmula para ambos.

2. ¿A qué se debe que la super clase Proyecto es una clase abstracta?

Porque tiene un método abstracto (calcularCostoActual) y porque representa un concepto general del cual no tiene sentido crear objetos directos con new.

3. ¿Cuál es la razón para diseñar e implementar una super clase abstracta sin métodos abstractos?

Para evitar que se creen instancias directas con new de esa clase, obligando a usar únicamente las clases hijas.

4. ¿Cómo se les conoce a las clases que sí pueden ser instanciadas?

Clases concretas.

5. ¿Cuál es la super clase por omisión en Java de cualquier clase diseñada o implementada por usted?

La clase Object (o java.lang.Object).

6. ¿Por qué razón la relación que existe entre la clase Proyecto y la clase Entregable es de agregación?

Porque el proyecto contiene entregables, pero si el proyecto se elimina o termina, los entregables siguen existiendo de forma independiente.

7. ¿Por qué razón la relación que existe entre la clase Proyecto y la clase Persona es de asociación?

Porque el proyecto solo guarda una referencia al encargado para conocer sus datos, sin que la persona sea parte física del proyecto ni dependa de él.

8. ¿Por qué razón la relación que existe entre la clase Persona y la clase Proyecto es de asociación?

Porque la persona guarda una referencia a su proyecto para consultar información (como su costo actual), manteniendo ciclos de vida separados.

9. ¿Para qué se utiliza la palabra super dentro de la implementación del método constructor de la clase Temporal?

Para ejecutar el constructor de la clase padre (Proyecto) e inicializar los datos comunes (nombre, costo base, fecha, código) antes de asignar los atributos propios.

10. ¿Para qué se utiliza la palabra super dentro de la implementación del método toString de la clase Temporal?

Para reutilizar el texto que genera el toString() de Proyecto y solamente añadirle los datos específicos del proyecto temporal.

11. ¿Cuáles son todas las condiciones para decir que en esta actividad asincrónica se presenta un ejemplo del concepto de sobre escritura de métodos?

Se deben cumplir tres condiciones:

- *Que el método ya tenga código programado en la clase padre.*
- *Que tenga exactamente el mismo nombre, tipo de retorno y parámetros.*
- *Que la clase hija llame al método del padre con super y le agregue comportamiento propio.*

12. ¿Por qué la implementación del método calcularCostoActual() en la clase Temporal NO materializa el concepto de sobreescritura de métodos? Fundamente su respuesta en las condiciones presentadas en la base teórica.

Porque incumple la primera condición: en la clase padre Proyecto el método no tiene código (es abstracto). Por tanto, no se está modificando un comportamiento previo, sino implementándolo por primera vez.

13. Provea dos ejemplos de contextos distintos donde sea absolutamente requerida al menos una relación de asociación, una relación de agregación y una jerarquía de herencia (una super clase y al menos dos subclases). Esos contextos deben ser escritos en prosa, en idioma español **y deben ser totalmente distintos a los contextos presentados en las actividades asincrónicas previas.**
    Luego, para CADA contexto diga lo siguiente:
    a. Tal Clase con Tal Clase se debe vincular con asociación debido a: _____
    b. Tal Clase con Tal Clase se debe vincular con agregación debido a: _____
    c. Tal Super Clase tiene Tales Subclases debido a: _____

Contexto 1: Taller de Bicicletas

Un taller da servicio y mantenimiento a diferentes tipos de bicicletas (de ruta y de montaña). Cada bicicleta tiene asignado un mecánico responsable de revisarla. Además, a una bicicleta se le pueden instalar accesorios (como luces o canastas), los cuales se pueden quitar o cambiar sin que dejen de existir si la bicicleta se da de baja del sistema.

- ***a. Bicicleta con Mecánico se debe vincular con asociación debido a:***

La bicicleta necesita conocer quién la repara y el mecánico sabe qué bicicleta atiende, pero ambos son independientes y tienen vidas separadas en el sistema.

- ***b. Bicicleta con Accesorio se debe vincular con agregación debido a:***

La bicicleta contiene accesorios, pero los accesorios no dependen de ella; si la bicicleta se retira, los accesorios siguen existiendo y se pueden reutilizar.

- ***c. La superclase Bicicleta tiene las subclases BicicletaRuta y BicicletaMontana debido a:***

Ambas son bicicletas y comparten datos básicos (marca y costo), pero cada una calcula su costo de mantenimiento de manera distinta por sus piezas especiales.

Contexto 2: Empresa de Envíos de Paquetes

Una empresa reparte paquetes que pueden ser de entrega estándar o de entrega exprés. Cada paquete es entregado por un repartidor asignado. Asimismo, existe una ruta de entrega que agrupa un conjunto de paquetes; si una ruta se cancela por mal clima, los paquetes siguen existiendo y se resignan a otra ruta.

- ***a. Paquete con Repartidor se debe vincular con asociación debido a:***

El paquete conoce al repartidor que lo transporta para registrar la entrega, pero ninguno depende de la existencia del otro.

- ***b. RutaEntrega con Paquete se debe vincular con agregación debido a:***

La ruta contiene una lista de paquetes para repartir, pero si la ruta se cancela o finaliza, los paquetes no desaparecen y siguen en el sistema.

- ***c. La superclase Paquete tiene las subclases PaqueteEstandar y PaqueteExpress debido a:***

Ambos son paquetes (comparten código, peso y destino), pero el paquete exprés calcula una tarifa adicional por entrega rápida.

D. Ejecute el programa PrincipalProyecto. ¿Usted obtuvo lo mismo que yo?

Tome una captura de pantalla de lo obtenido por usted.

EN SU CAPTURA DE PANTALLA DEBE SALIR SU NOMBRE

Tenga presente que la fecha de inicio de cada proyecto corresponderá a la fecha en que usted ejecute el programa.

```
Estudiante: Brandon Alvarez Franco
Código del Proyecto: PRY-001
Nombre del Proyecto: Implementación ERP
Costo inicial: 100.0
Fecha de inicio: 2026-10-25
Ingresos acumulados: 3600.0
Costo actual: 1000.0
Balance: 2600.0
Encargado: Identificación = 123456 Nombre Completo = Leopoldo Octavio de Montferrat y Cifuentes

Entregables (2):
- Nombre del entregable: Módulo Financiero Estado: A TIEMPO

- Nombre del entregable: Módulo Recursos Humanos Estado: RETRASADO

Tipo de proyecto: Temporal
Duración en semanas: 12
Valor del dólar: 10.0

Código del Proyecto: PRY-002
Nombre del Proyecto: Mantenimiento Infraestructura
Costo inicial: 200.0
Fecha de inicio: 2026-10-25
Ingresos acumulados: 800.0
Costo actual: 255.0
Balance: 545.0
Encargado: Identificación = 7891011 Nombre Completo = Isadora Celeste de Alencourt y Villamediana

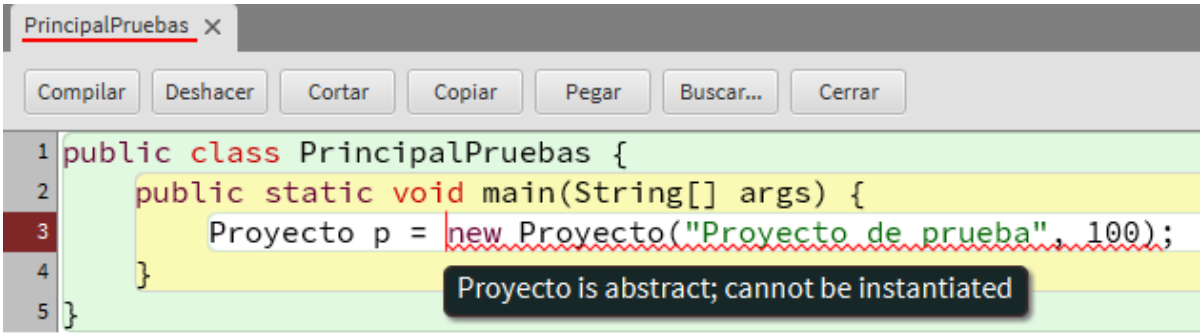
Entregables (1):
- Nombre del entregable: Revisión Servidores Estado: A TIEMPO

Tipo de proyecto: Permanente
Costo anual fijo: 50.0
Porcentaje de inflación: 0.1

La siguiente persona: Identificación = 123456 Nombre Completo = Leopoldo Octavio de Montferrat y Cifuentes
 es encargado-a de un proyecto cuyo costo actual es: 1000.0
La siguiente persona: Identificación = 7891011 Nombre Completo = Isadora Celeste de Alencourt y Villamediana
 es encargado-a de un proyecto cuyo costo actual es: 255.0
```

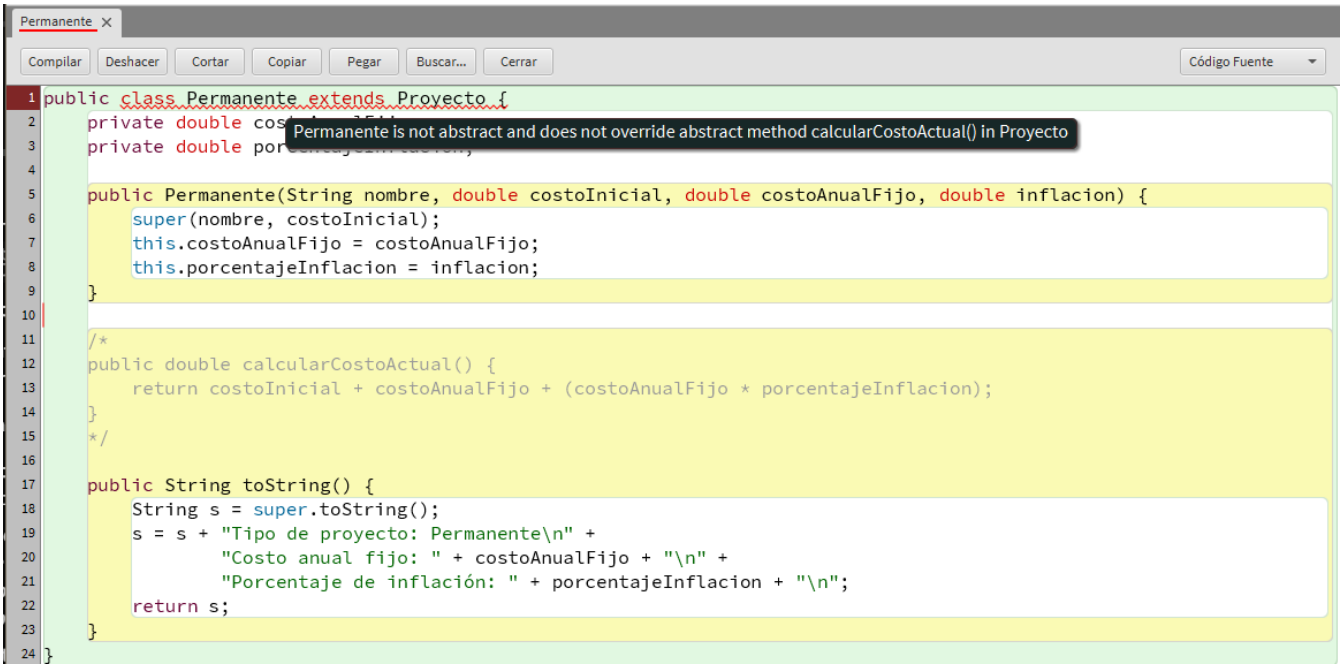
E. **Experimentos con la jerarquía de herencia.** En este punto usted va a provocar, de forma deliberada, incumplimientos de la jerarquía propuesta para observar cómo responde el compilador. Para cada uno de los siguientes experimentos realice únicamente la modificación indicada, compile, tome una captura de pantalla del mensaje de error que reporta BlueJ y responda con sus propias palabras las preguntas del experimento. **Al finalizar cada experimento restaure el código a su versión original, tal como se presenta en esta actividad asincrónica, antes de iniciar el siguiente.**

1. Agregue a su PrjProyecto una clase llamada PrincipalPruebas, con un método main en el que se declare un objeto de tipo Proyecto y se instancie de la siguiente forma:
   `Proyecto p = new Proyecto("Proyecto de prueba", 100);`
   ¿Por qué Java no permite construir ese objeto? Al finalizar este experimento elimine la clase PrincipalPruebas.



Porque la clase Proyecto está definida como abstracta (abstract). En Java, las clases abstractas sirven únicamente como plantilla general para sus clases hijas y tienen prohibido crear objetos directamente con new.

2. En la clase Permanente, convierta en comentario la implementación completa del método calcularCostoActual(). ¿Por qué la clase Permanente deja de compilar? ¿Qué alternativa tendría la clase Permanente para compilar sin implementar ese método y qué consecuencia tendría esa alternativa sobre la clase PrincipalProyecto?




```
public class Permanente extends Proyecto {
    private double cos
    private double por

    public Permanente(String nombre, double costoInicial, double costoAnualFijo, double inflacion) {
        super(nombre, costoInicial);
        this.costoAnualFijo = costoAnualFijo;
        this.porcentajeInflacion = inflacion;
    }

    /*
    public double calcularCostoActual() {
        return costoInicial + costoAnualFijo + (costoAnualFijo * porcentajeInflacion);
    }
    */

    public String toString() {
        String s = super.toString();
        s = s + "Tipo de proyecto: Permanente\n" +
                "Costo anual fijo: " + costoAnualFijo + "\n" +
                "Porcentaje de inflación: " + porcentajeInflacion + "\n";
        return s;
    }
}
```

¿Por qué la clase Permanente deja de compilar?

Porque es una clase concreta y está obligada a darle código a todos los métodos abstractos que hereda de Proyecto. Al comentarlo, queda incompleta.

¿Qué alternativa tendría la clase Permanente para compilar sin implementar ese método?

Declarar la clase Permanente también como abstracta (public abstract class Permanente extends Proyecto).

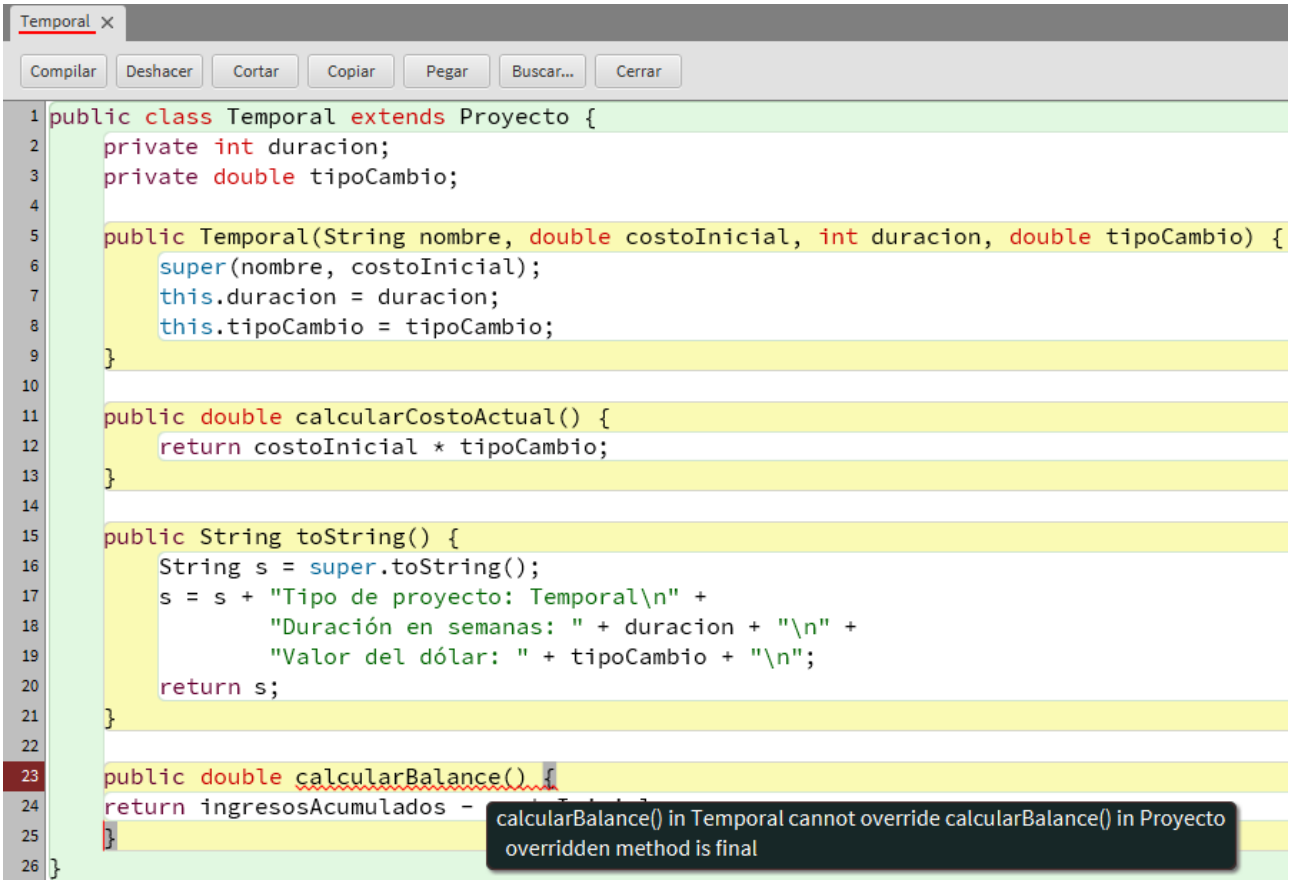
¿Qué consecuencia tendría esa alternativa sobre la clase PrincipalProyecto?

Que en PrincipalProyecto daría error al intentar crear el objeto con new Permanente(...), ya que las clases abstractas no se pueden instanciar.

3. En la clase Temporal, agregue el siguiente método:

```
public double calcularBalance() {
    return ingresosAcumulados - costoInicial;
}
```

¿Por qué la clase Temporal deja de compilar? ¿Por qué esa restricción es coherente con el contexto de Proyectos?



¿Por qué la clase Temporal deja de compilar?

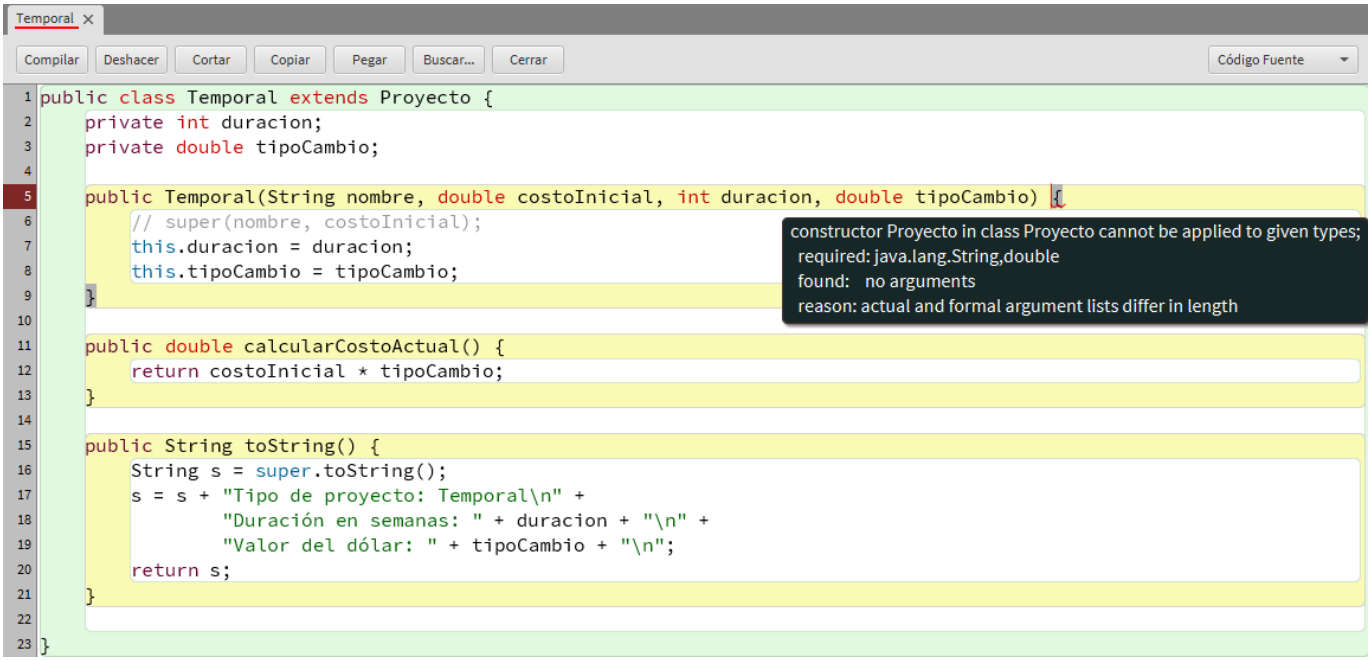
Porque en la clase padre Proyecto el método calcularBalance() tiene la palabra final, lo que prohíbe que cualquier clase hija lo cambie o sobreescriba.

¿Por qué esa restricción es coherente con el contexto de Proyectos?

Porque la fórmula para calcular el balance (ingresos menos costos) es una regla fija de la empresa que debe ser igual para todos los proyectos, sin excepciones.

4. En el constructor de la clase Temporal, convierta en comentario la instrucción:
   `super(nombre, costoInicial);`

¿Por qué el compilador reporta un error relacionado con el constructor de la clase Proyecto, si usted no lo está invocando? ¿Por qué la clase Proyecto no cuenta con un constructor sin parámetros?



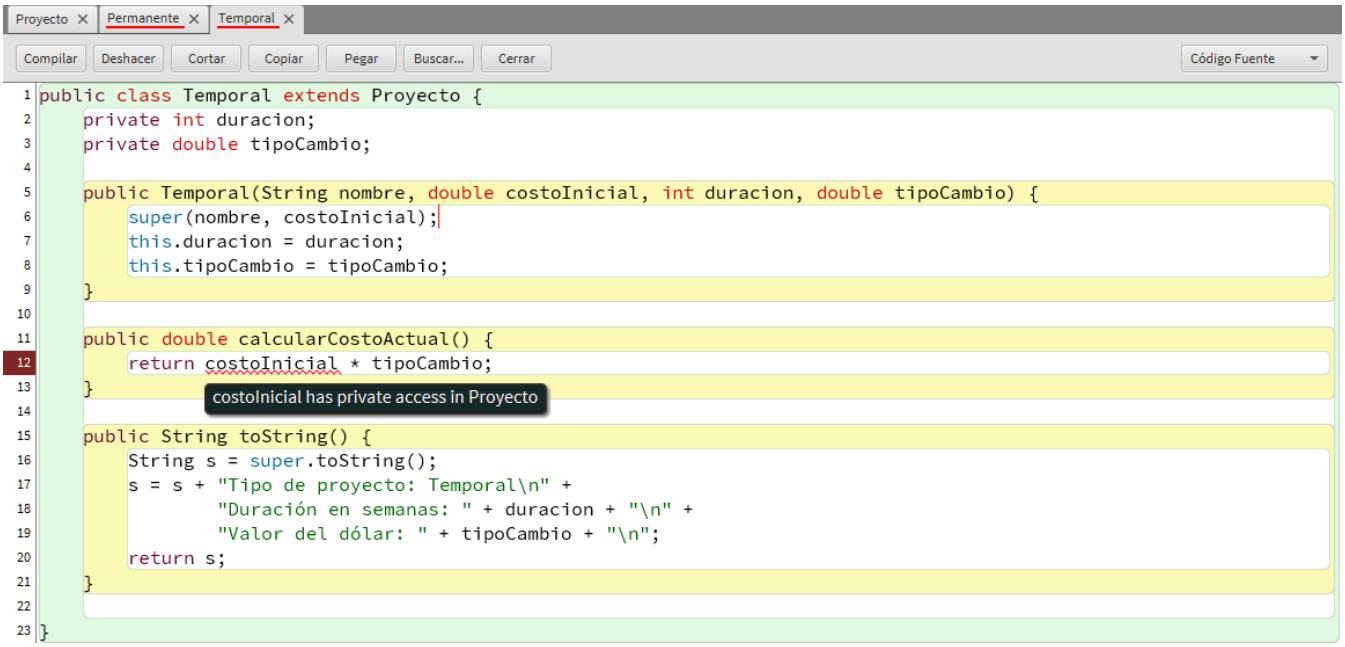
¿Por qué el compilador reporta un error con el constructor de Proyecto, si no lo estamos invocando?

Porque si no escribes super(...) manualmente, Java intenta llamar de forma automática a un constructor vacío sin parámetros (super()) en la clase padre.

¿Por qué la clase Proyecto no cuenta con un constructor sin parámetros?

Porque al haber creado nosotros un constructor con dos parámetros (nombre y costoInicial), Java retira automáticamente el constructor vacío por defecto.

5. En la clase Proyecto, cambie el modificador de acceso del atributo costoInicial de protected a private. ¿Por qué las subclases Temporal y Permanente dejan de compilar? ¿Puede una super clase con atributos privados permitir que el valor de esos atributos sea utilizado desde sus subclases? ¿De qué forma?



¿Por qué las subclases Temporal y Permanente dejan de compilar?

Porque al ser private, el atributo solo se puede usar dentro de Proyecto. Las clases hijas no tienen permiso para leerlo ni modificarlo directamente.

¿Puede una superclase con atributos privados permitir que el valor de esos atributos sea utilizado desde sus subclases? ¿De qué forma?

Sí. Se logra creando un método público o protegido para obtener el valor (un getter, como getCostoInicial()).

F. Algo se puede mejorar en la programación propuesta de la clase Proyecto. Observe con detenimiento esa clase y note las siguientes dos situaciones:

- El prefijo del código del proyecto ("PRY-") está quemado en el código como valor inicial del atributo codigo. Recuerde cómo, en la actividad asincrónica anterior, la clase Edificio incorporó el prefijo de su código.
- En la base teórica se estableció que, en un contexto de herencia, si los atributos de la super clase requieren ser accedidos desde la subclase el modificador de acceso debe ser protected, y private en caso contrario. Sin embargo, por razones didácticas, en la clase Proyecto la mayoría de los atributos quedaron con el modificador de acceso protected.

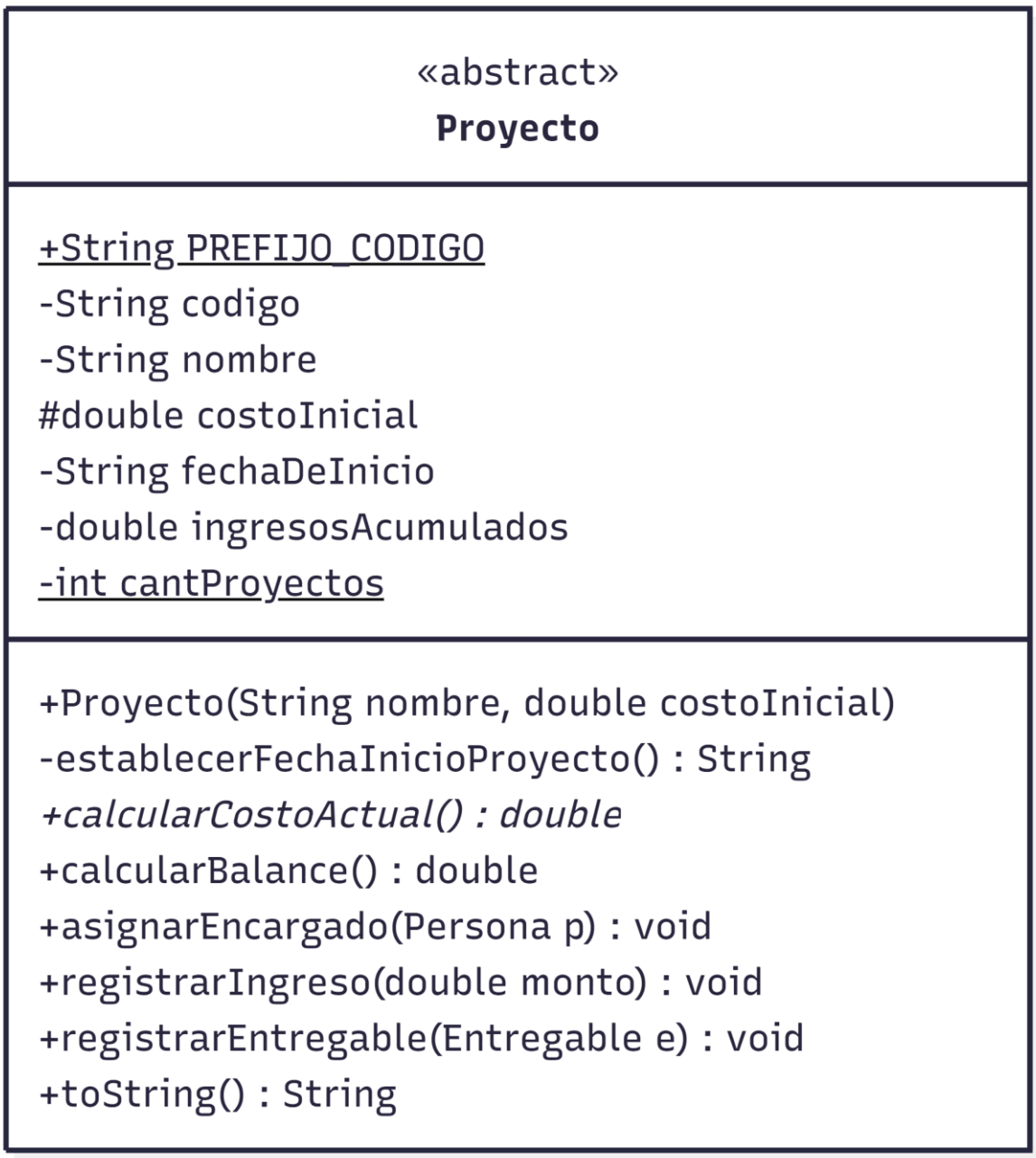
Atributos que quedaron con modificador protected y justificación:

El único atributo que se dejó en protected fue ***costoInicial****.*

Se requiere así porque las dos subclases lo usan directamente dentro de su método calcularCostoActual():

- *Temporal lo usa para multiplicar por el tipo de cambio (costoInicial * tipoCambio).*
- *Permanente lo usa como base para sumarle el costo fijo y la inflación.*

Todos los demás atributos pasaron a private porque las clases hijas no necesitan acceder a ellos directamente.


«abstract»
Proyecto
+String PREFIJO_CODIGO
-String codigo
-String nombre
#double costoInicial
-String fechaDeInicio
-double ingresosAcumulados
-int cantProyectos
+Proyecto(String nombre, double costoInicial)
-establecerFechaInicioProyecto() : String
+calcularCostoActual() : double
+calcularBalance() : double
+asignarEncargado(Persona p) : void
+registrarIngreso(double monto) : void
+registrarEntregable(Entregable e) : void
+toString() : String

```
Estudiante: Brandon Alvarez Franco
Código del Proyecto: PRY-001
Nombre del Proyecto: Implementación ERP
Costo inicial: 100.0
Fecha de inicio: 2026-10-25
Ingresos acumulados: 3600.0
Costo actual: 1000.0
Balance: 2600.0
Encargado: Identificación = 123456 Nombre Completo = Leopoldo Octavio de Montferrat y Cifuentes

Entregables (2):
- Nombre del entregable: Módulo Financiero Estado: A TIEMPO

- Nombre del entregable: Módulo Recursos Humanos Estado: RETRASADO

Tipo de proyecto: Temporal
Duración en semanas: 12
Valor del dólar: 10.0

Código del Proyecto: PRY-002
Nombre del Proyecto: Mantenimiento Infraestructura
Costo inicial: 200.0
Fecha de inicio: 2026-10-25
Ingresos acumulados: 800.0
Costo actual: 255.0
Balance: 545.0
Encargado: Identificación = 7891011 Nombre Completo = Isadora Celeste de Alencourt y Villamediana

Entregables (1):
- Nombre del entregable: Revisión Servidores Estado: A TIEMPO

Tipo de proyecto: Permanente
Costo anual fijo: 50.0
Porcentaje de inflación: 0.1

La siguiente persona: Identificación = 123456 Nombre Completo = Leopoldo Octavio de Montferrat y Cifuentes
 es encargado-a de un proyecto cuyo costo actual es: 1000.0
La siguiente persona: Identificación = 7891011 Nombre Completo = Isadora Celeste de Alencourt y Villamediana
 es encargado-a de un proyecto cuyo costo actual es: 255.0
```

K. Ejecute el programa PrincipalCasetaPeaje. ¿Usted obtuvo lo mismo que yo?

Tome una captura de pantalla de lo obtenido por usted.

```
Estudiante: Brandon Alvarez Franco
La primera caseta:
Caseta de Peaje número: 1
Autopista= Ruta 32 al Caribe de Costa Rica
Total Cobrado= 375.0

La segunda caseta:
Caseta de Peaje número: 2
Autopista= Ruta 32 al Caribe de Costa Rica
Total Cobrado= 775.0

Casetas creadas: 2
Primer vehículo creado
Placa= ABC123
Tipo= particular
Marca= Toyota
Modelo= Corolla
Año= 2018
```

O. **Responda en el documento creado en el punto A las siguientes preguntas**:

1. ¿Qué ocurrió con la instrucción switch del método cobrarPeaje? ¿Cómo sabe ahora el objeto de tipo CasetaPeaje cuánto debe cobrar a cada vehículo?

El switch se eliminó por completo. La caseta ya no revisa qué tipo de vehículo es; únicamente llama a obj.calcularMontoPeaje() y cada vehículo devuelve su propio monto automáticamente gracias al polimorfismo.

2. ¿Por qué la firma del método cobrarPeaje no tuvo que cambiar, a pesar de que ahora existen cuatro subclases de Vehiculo?

Porque el método sigue recibiendo un parámetro de tipo Vehiculo (Vehiculo obj). Como las cuatro subclases (Particular, CargaLiviana, Carga, Moto) son hijas de

Vehiculo, Java permite pasar cualquier objeto de estas clases sin alterar la firma.

3. ¿Por qué la declaración Vehiculo a, b, c, d, e, f, g; del programa principal se mantuvo intacta? Relacione su respuesta con la regla del curso: siempre se declara del super tipo y se instancia del subtipo.

Se mantuvo intacta porque se aplica la regla de declarar la variable usando la superclase (Vehiculo) e instanciar con la subclase específica (como new Moto(...)). Esto permite manipular todos los vehículos de forma uniforme.

4. ¿Por qué el método calcularMontoPeaje() debe ser abstracto en la clase Vehiculo?

Porque no existe un monto general para un vehículo genérico; cada tipo cobra una tarifa diferente. Al declararlo abstracto, se obliga a que cada clase hija implemente su propio cálculo.

5. ¿Qué decisión tomó con respecto al atributo tipo de la clase Vehiculo y cómo logró que el estado de cada vehículo siga indicando su tipo? Justifique su decisión.

Se mantuvo el atributo tipo en Vehiculo y su valor se envía desde el constructor de cada subclase usando super(..., TIPO, ...). Así se evita duplicar código y el método toString() original sigue mostrando el tipo de vehículo sin cambios.

6. ¿Qué atributos de la clase Vehiculo quedaron con el modificador de acceso protected y cuáles con private? Justifique cada decisión.

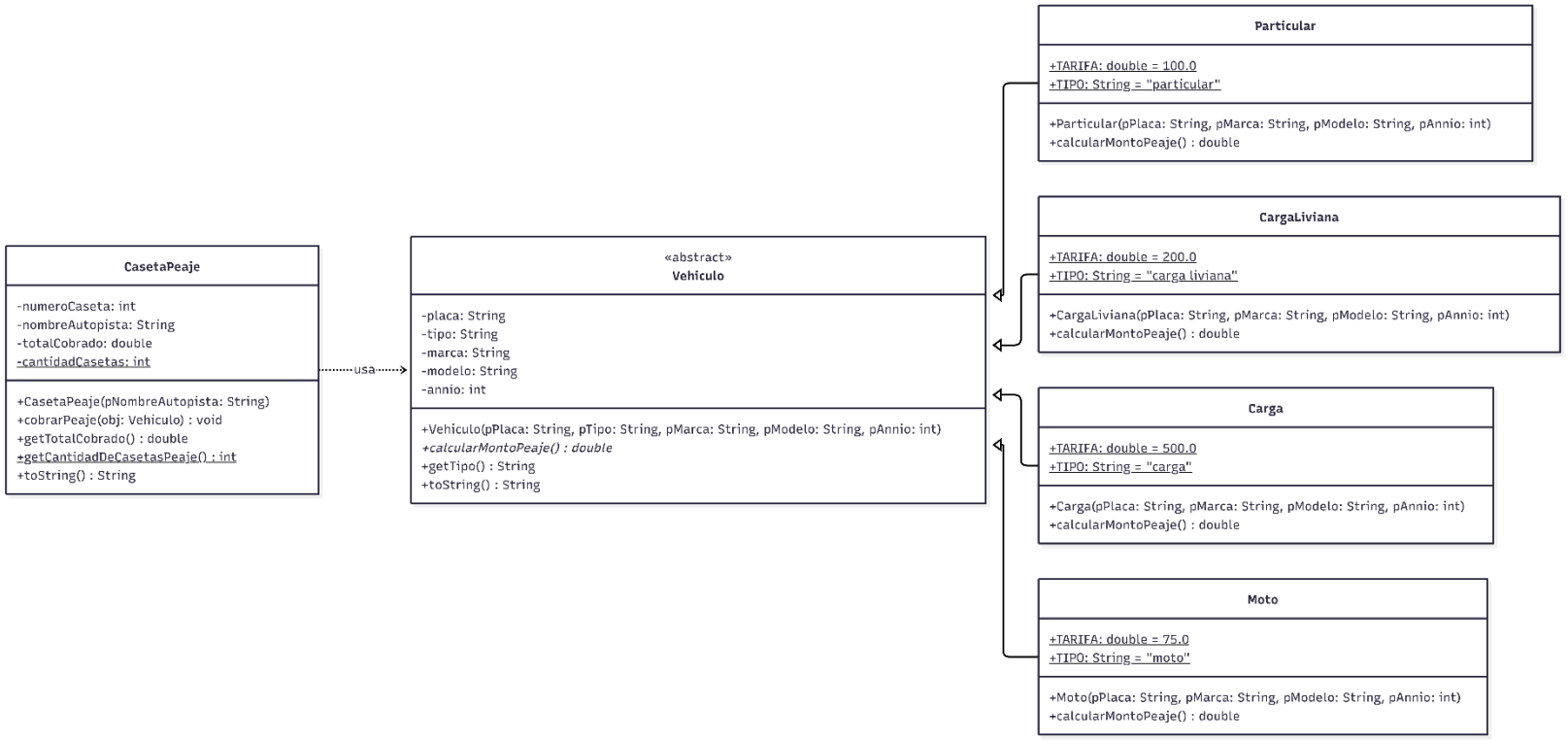
*Todos los atributos (placa, tipo, marca, modelo, annio) quedaron en **private**. Ninguno requirió ser protected porque las clases hijas no los leen ni modifican directamente; todo se gestiona a través del constructor y los métodos de la clase padre.*

7. Suponga que la autopista decide cobrar peaje a un nuevo tipo de vehículo: los autobuses. ¿Qué clases tendría que agregar o modificar en su diseño? ¿Tendría que modificar la clase CasetaPeaje? Justifique.

Solo se debe crear una nueva clase Autobus que herede de Vehiculo y devuelva su tarifa en calcularMontoPeaje(). No se modifica CasetaPeaje, ya que su método puede cobrar a cualquier vehículo nuevo sin alterar su código.

8. ¿Qué tipo de relación existe entre CasetaPeaje y Vehiculo? ¿Cambió esa relación con respecto a la AA6? Justifique.

*Es una relación de **dependencia** (de uso). **No cambió**, sigue siendo exactamente la misma porque CasetaPeaje no almacena vehículos como atributos; únicamente recibe un Vehiculo como parámetro temporal para cobrarle.*

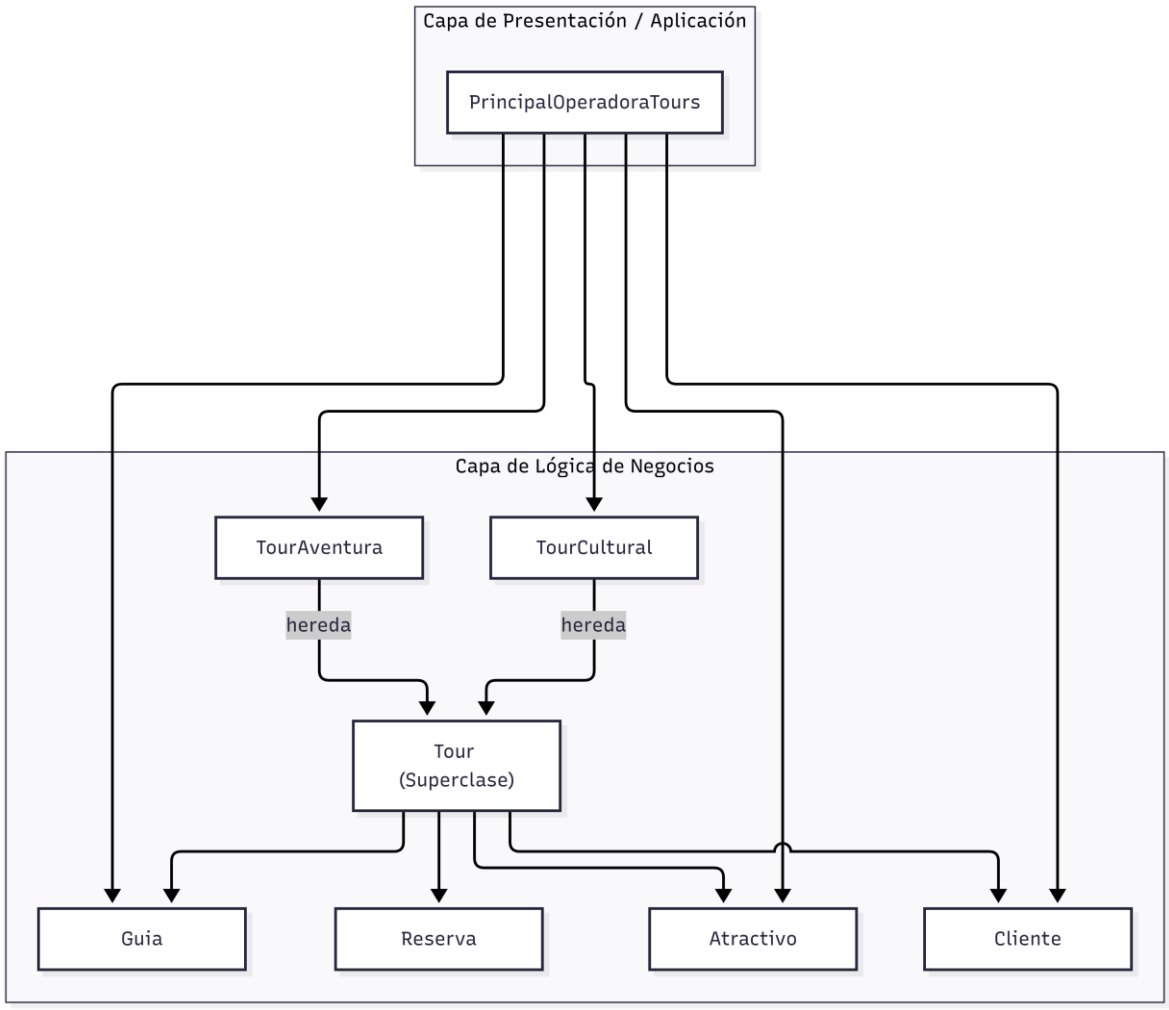



```
Estudiante: Brandon Alvarez Franco
La primera caseta:
Caseta de Peaje número: 1
Autopista= Ruta 32 al Caribe de Costa Rica
Total Cobrado= 375.0

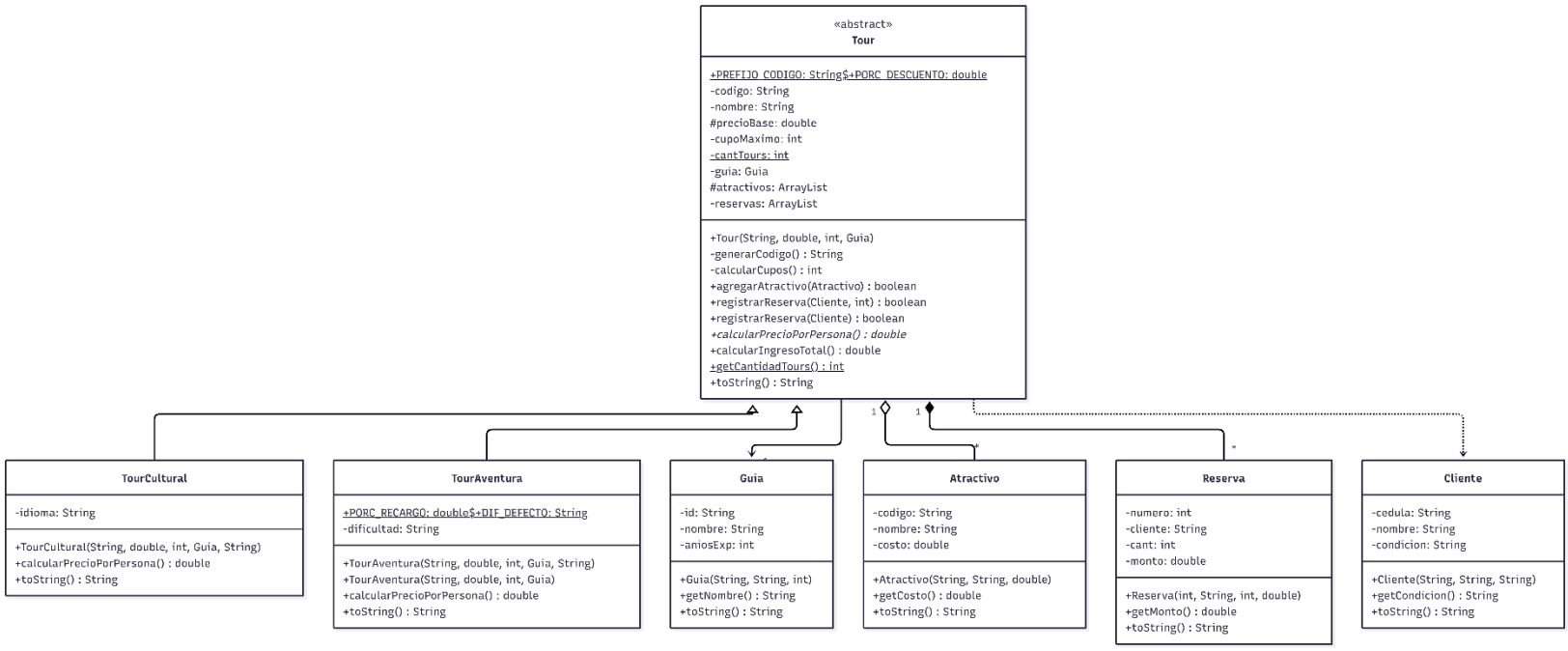
La segunda caseta:
Caseta de Peaje número: 2
Autopista= Ruta 32 al Caribe de Costa Rica
Total Cobrado= 775.0

Casetas creadas: 2
Primer vehículo creado
Placa= ABC123
Tipo= particular
Marca= Toyota
Modelo= Corolla
Año= 2018
```

R. Proponga un **diagrama de arquitectura conceptual** que atienda el contexto anterior.



S. Proponga un **diagrama de clase con detalles de implementación** que atienda el contexto anterior. Asegúrese de que el diagrama cumpla con la notación UML tal como se ha abordado en las actividades asincrónicas del curso. En particular:



V. Ejecute el programa PrincipalOperadoraTours **sin modificarlo**, excepto la primera instrucción, en la que debe escribir su nombre completo. Tome una captura de pantalla con la evidencia completa de la ejecución. **EN SU CAPTURA DE PANTALLA DEBE SALIR SU NOMBRE.**

```
Estudiante: Brandon Alvarez Franco
================================================
--- INFORMACIÓN DEL TOUR CULTURAL ---
Código: TOUR-001
Tour: Ruta Histórica San José
Precio Base: 12000.0
Cupo Máximo: 15 (Disponibles: 11)
Guía asignado: Carlos Murillo Solano (ID: G-101, Exp: 8 años)
Atractivos (2):
  - Museo Nacional [Cód: ATR-01, Entrada: 3000.0]
  - Teatro Nacional [Cód: ATR-02, Entrada: 4500.0]
Reservas registradas (2):
  - Reserva #1 - Sofía Rojas Blanco | Personas: 3 | Total pagado: 46800.0
  - Reserva #2 - John Miller | Personas: 1 | Total pagado: 19500.0
Precio por persona: 19500.0
Ingreso total acumulado: 66300.0
Tipo: Cultural
Idioma: Español

--- INFORMACIÓN DEL TOUR DE AVENTURA ---
Código: TOUR-002
Tour: Expedición Arenal Extremo
Precio Base: 25000.0
Cupo Máximo: 6 (Disponibles: 3)
Guía asignado: Elena Vega Chaves (ID: G-102, Exp: 5 años)
Atractivos (1):
  - Catarata La Fortuna [Cód: ATR-03, Entrada: 6000.0]
Reservas registradas (2):
  - Reserva #1 - John Miller | Personas: 2 | Total pagado: 57499.99999999999
  - Reserva #2 - Sofía Rojas Blanco | Personas: 1 | Total pagado: 23000.0
Precio por persona: 28749.999999999996
Ingreso total acumulado: 80500.0
Tipo: Aventura
Dificultad: alto

================================================
Total de tours registrados en la operadora: 2
```

W. **Responda en el documento creado en el punto A las siguientes preguntas**:

1. ¿Por qué la clase Tour debe ser una super clase abstracta? Justifique su decisión a partir del contexto.

Porque el contexto dice que no existe ningún tour genérico (todos son o culturales o de aventura), y porque cada tipo calcula su precio por persona con una fórmula distinta. Al hacerla abstracta, se impide instanciarla con new y se obliga a usar las clases hijas.

2. ¿Qué métodos abstractos incorporó en la clase Tour? ¿Por qué la implementación de cada uno corresponde a cada subclase?

Método incorporado: *public abstract double calcularPrecioPorPersona();*

Porque la regla de cobro cambia según el tipo de tour: en TourCultural se suma el precio base más las entradas de sus atractivos, mientras que en TourAventura se suma el precio base más un 15 % de recargo por seguro.

3. ¿Qué métodos declaró como final y cómo los representó en su diagrama? Justifique su decisión a partir del contexto.

Método: *public final double calcularIngresoTotal()*

Diagrama UML: *Se coloca la etiqueta {leaf} al final del método: + calcularIngresoTotal(): double {leaf}.*

Justificación: *El requerimiento indica textualmente que la fórmula para sumar los ingresos de las reservas aplica igual para todos los tours y ningún tipo de tour puede modificarla.*

4. ¿Qué atributos de la clase Tour quedaron con el modificador de acceso protected y cuáles con private? Justifique cada decisión a partir de la programación de sus subclases.

Atributos protected: *precioBase y atractivos. Se dejaron protegidos porque la subclase TourCultural necesita leerlos directamente para calcular el precio por persona sumando las entradas.*

Atributos private: *codigo, nombre, cupoMaximo, guia, reservas y cantTours. Se dejaron privados porque las clases hijas no necesitan manipularlos directamente; se gestionan con los métodos de la clase padre.*

5. ¿En qué métodos de sus subclases se materializa el concepto de sobreescritura de métodos? Demuestre que se cumplen todas las condiciones.

Se materializa en el método ***toString()*** *de TourCultural y TourAventura. Cumple las 3 condiciones:*

1. *El método ya existe con código implementado en la clase padre Tour.*
2. *Tiene la misma firma (mismo nombre, sin parámetros y retorna String).*
3. *La subclase invoca super.toString() y le concatena sus propios atributos (el idioma o la dificultad).*

6. ¿Cómo utilizó la palabra reservada super en los constructores de sus subclases? ¿Cómo implementó la sobrecarga de constructores de la clase TourAventura y cómo evitó la repetición innecesaria de código entre ellos?

Uso de super: *Se usa en la primera línea del constructor de cada clase hija para llamar al constructor padre: super(nombre, precioBase, cupoMaximo, guia); e inicializar los datos comunes.*

Sobrecarga en TourAventura: *Se hicieron dos constructores: uno que recibe la dificultad indicada por el usuario y otro que no la recibe.*

Evitar repetición: *El constructor sin dificultad no repite código; llama al constructor completo usando this(nombre, precioBase, cupoMaximo, guia, DIFICULTAD_POR_OMISION).*

7. ¿Qué tipo de relación estableció entre cada uno de los siguientes pares de clases? Justifique cada decisión, su navegabilidad y, cuando corresponda, su multiplicidad a partir del contexto:
   a. Tour y Guia.
   b. Tour y Atractivo.
   c. Tour y la clase con la que representó las reservas.
   d. Tour y Cliente.

a) ***Tour y Guia:***

- ***Relación y navegabilidad:*** *Asociación unidireccional de Tour hacia Guia.*
- ***Multiplicidad:*** *Un Tour tiene 1 Guía (1), y una Guía puede estar en muchos Tours (*).*
- ***Justificación:*** *El tour conoce a su guía para consultar su nombre, pero la guía no necesita conocer los tours que dirige ni depende de ellos.*

b) ***Tour y Atractivo:***

- ***Relación y navegabilidad:*** *Agregación unidireccional de Tour hacia Atractivo (rombo blanco).*
- ***Multiplicidad:*** *Un Tour tiene de 0 a muchos Atractivos (0..*), y un Atractivo puede estar en varios tours (*).*
- ***Justificación:*** *El tour contiene atractivos en su recorrido, pero los atractivos existen de forma independiente en el sistema antes y después de los tours.*

c) ***Tour y Reserva:***

- ***Relación y navegabilidad:*** *Composición unidireccional de Tour hacia Reserva (rombo negro).*
- ***Multiplicidad:*** *1 Tour contiene de 0 a muchas Reservas (0..*).*
- ***Justificación:*** *El contexto indica que una reserva no tiene sentido fuera de su tour y, si el tour deja de existir, sus reservas también dejan de existir.*

d) ***Tour y Cliente:***

- ***Relación y navegabilidad:*** *Dependencia unidireccional de Tour hacia Cliente.*

- ***Justificación:*** *El tour no guarda clientes como atributos; solo recibe a Cliente como parámetro en el método registrarReserva para leer su nombre y aplicar el descuento si es nacional.*

8. ¿Cómo garantizó que las reservas de cada tour se numeren a partir de 1, sin importar las reservas registradas en los demás tours? ¿Por qué ese número no puede obtenerse a partir de un atributo de clase?

Garantía: *El número de reserva se calcula con el tamaño de la lista de reservas de ese tour más uno (reservas.size() + 1). Al ser una lista propia de cada tour, cada uno inicia en 1 independientemente.*

Por qué no de clase: *Porque un atributo de clase (static) es global y compartido por todos los tours del programa. Si fuera estático, todas las reservas compartirían el mismo contador y no empezarían en 1 en cada tour.*

9. ¿Qué atributos y métodos de clase incorporó en su diseño? ¿Por qué deben ser de clase y no de instancia?

Atributo y método: *El atributo cantTours y el método getCantidadTours() en la clase Tour.*

Por qué de clase: *Porque el total de tours registrados le pertenece al sistema en general. Si fueran de instancia, cada tour tendría un contador por separado y no se podría saber el gran total acumulado.*

10. ¿Qué constantes incorporó en su diseño y qué valores quemados evitó con ellas?

a) ***En Tour:***

- *PREFIJO_CODIGO = "TOUR-" (evita quemar el texto del código).*
- *PORCENTAJE_DESCUENTO_NACIONAL = 0.20 (evita quemar el 20 % de descuento).*
- *CONDICION_NACIONAL = "nacional" (evita quemar la condición en texto).*

b) ***En TourAventura:***

- *PORCENTAJE_RECARGO = 0.15 (evita quemar el recargo del 15 %).*

- *DIFICULTAD_POR_OMISION = "medio" (evita quemar la dificultad por defecto).*

11. ¿En qué métodos, además de los constructores, aplicó sobrecarga? ¿Cómo evitó la repetición innecesaria de código entre los métodos sobrecargados?

Método sobrecargado: *En registrarReserva dentro de Tour.*

Cómo evitó repetición: *Se crearon dos versiones: una con cantidad de personas y otra que solo recibe el cliente. La versión de un solo parámetro simplemente invoca a la completa pasándole un 1 (this.registrarReserva(cliente, 1);), evitando duplicar validaciones.*

12. Enumere cada uno de los métodos privados que incorporó y explique por qué cada uno DEBE ser privado.

generarCodigo() en Tour: *Crea el código TOUR-X. Debe ser privado para que nadie externo pueda alterar el identificador único del tour.*

existeAtractivo(Atractivo a) en Tour: *Verifica si un atractivo ya está en la lista. Debe ser privado porque es una validación interna exclusiva del método agregarAtractivo.*

calcularCuposDisponibles() en Tour: *Calcula cuántos espacios quedan restando personas reservadas al cupo máximo. Debe ser privado para encapsular la fórmula de validación antes de aceptar una reserva.*

13. Suponga que la operadora incorpora un nuevo tipo de tour: los tours gastronómicos, cuyo precio por persona corresponde al precio base más un monto fijo por concepto de degustación. ¿Qué tendría que agregar o modificar en su diseño? ¿Tendría que modificar la clase Tour o la clase PrincipalOperadoraTours? Justifique.

Qué agregar: *Solo se debe crear una nueva clase TourGastronomico que herede de Tour, agregue el atributo del costo de degustación e implemente su propio método calcularPrecioPorPersona().*

Modificar Tour: No se modifica*, porque gracias a la herencia y polimorfismo la clase base ya gestiona reservas, guías e ingresos sin importar el tipo de tour.*

Modificar PrincipalOperadoraTours: No es obligatorio modificarla *para que el sistema funcione; solo se modificaría si se desea probar e instanciar este nuevo tipo de tour.*